Projet : INFO0074-Groupe43

DUMOULIN MAXIME S160180

http://s160180.php2.hec.ulg.ac.be/groupe43/

Table des matières :

* Phase d'introduction : La base du projet ---------- 2
* Les hypothèses de travail : PHP, si puissant que ça ? ---------- 4
* La phase de commencement ---------- 5
    1) La base CSS ---------- 5
    2) La base HTML ---------- 6
* La programmation ---------- ---------- 8
    1) La base de données ---------- 8
    2) Liaison PHP/MySQL ---------- 14
    3) Les applications : ajouter, modifier et supprimer ---------- 16
* Les analyses statistiques : explications ---------- 19
* La conclusion du travail ---------- 21
* Bibliographie ---------- 21

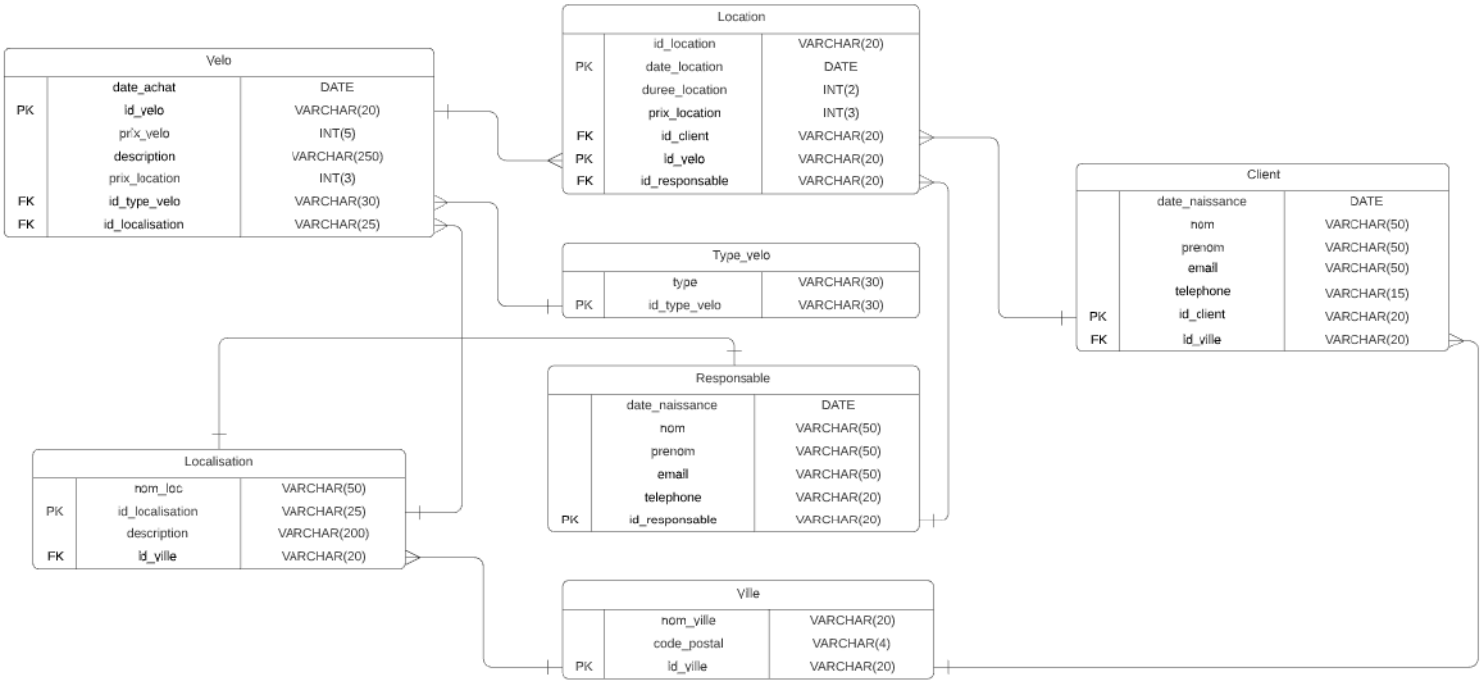
1) Phase d'introduction : La base du projet

Tout d'abord, il nous a été demander de réaliser un système digital en HTML, CSS, PHP, MySQL pour BikeRental, une PME installée en Belgique, qui propose des services de location de vélos aux touristes ou cyclistes occasionnels dans les grandes villes de Belgique.

Pour cela, j'ai commencé à parcourir différents sites de locations de vélos sur internet pour me donner une idée plus concrète de ce qui était demandé. Après, la consultation de divers sites, j'ai pu avoir une idée plus précise de ce qui m'était convié de réaliser.

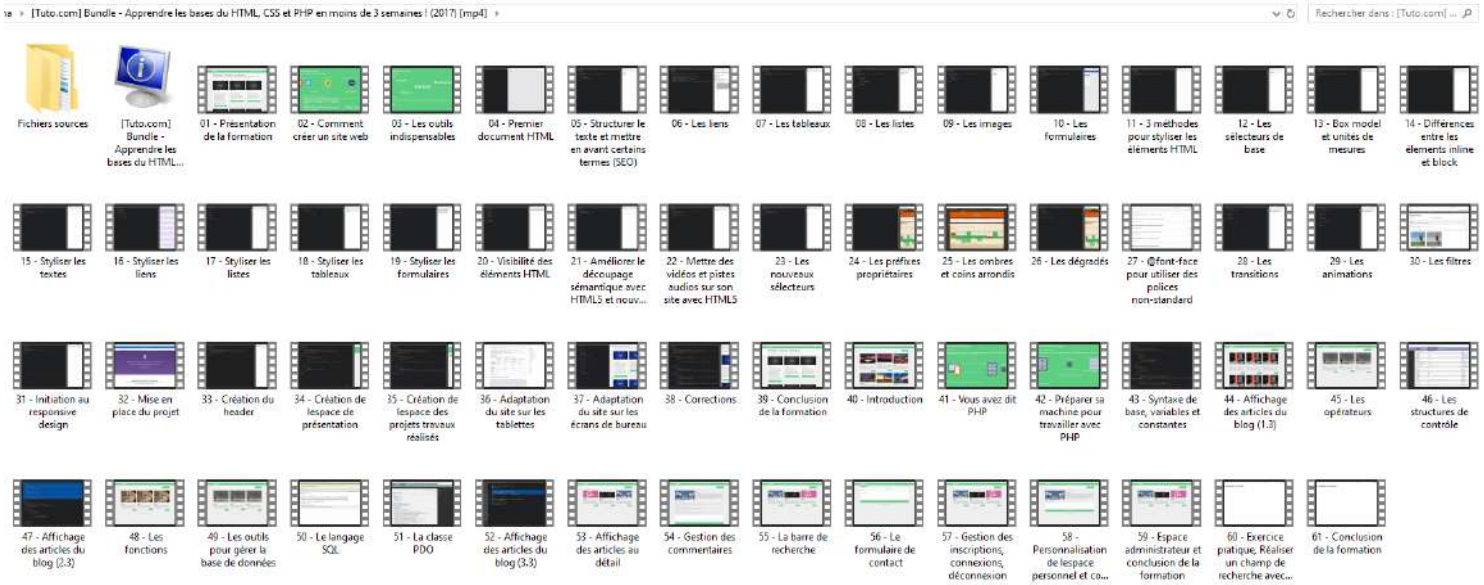
Après cela, j'ai commencé l'élaboration du codage, ce fût ma première erreur lors de ce travail. J'ai directement pu constater différentes incohérences par la suite (lorsque j'ai dû lier mon code avec PHP MyAdmin).

À la suite de mon erreur, j'ai créé un modèle assez brouillon des entités-relations de BikeRental via « Lucid Chart » pour avoir une meilleure base de travail, ce que j'ai utilisé par la suite comme les fondations de mon système.



Ce dernier me servira à la construction même des différentes tables et entités de la base de données sur PHP MyAdmin ainsi que de la programmation à proprement parler.

Étant donné que je voulais avoir fini ce projet le plus tôt possible (pour éviter de le finir pendant mes examens), j'ai commencé directement en début d'année avant les différentes explications ou les podcasts de la matière. J'ai donc suivi une formation, sur internet. Je vous mets ci-dessous une capture d'écran du dossier de la formation (comme le dossier pèse 1.5Go je ne l'ajouterai pas à l'annexe).



Cependant cette formation n'abordait pas trop le langage MySQL, j'ai donc attendu les informations concernant cette partie.

Je tiens à préciser qu'aucun code n'a été « copier-coller » de cette formation, si besoin, je peux ultérieurement vous faire parvenir le dossier via un Cloud.

2) Les hypothèses de travail : PHP, si puissant que ça ?

Ayant une connaissance assez limitée mais basique en informatique, je n'avais aucune idée de la signification de cette extension « PHP » avant mi-février de cette année. Au fur et à mesure des jours j'ai appris à la connaître. Au début du travail, je ne voyais pas le rapport avec l'économie mais c'est vers la fin que tout a pris du sens avec les analyses statistiques.

Au début du travail, je ne me rendais pas compte de l'étendue de sa puissance, j'ai pu découvrir un nombre incalculable de formules mathématique sur le site développeur de PHP. Cela m'a montré qu'il était possible de faire plus que des simples sommes ou additions… J'ai pu découvrir par moi-même des formules statistique « stats_...._.... » plus qu'intéressant comme « stats_cdf_poisson » qui permet de calculer n'importe quel paramètre de la loi de Poisson avec des valeurs de son choix (lien avec le cours de Proba/Stat). Cependant, je ne les ai pas utilisé car elle me semblait un peu trop complexe pour ce travail mais il est sûr qu'elles pourront être très utiles dans un cours de statistique informatique. J'ai pu aussi découvrir une formule d'inversion matricielle (Lapack::pseudoInverse) et de résolution d'un système d'équations linéaires (Lapack::solveLinearEquation) qui peuvent être très utile pour le cours de math au Q1. Je pourrais vous donner bien d'autres formules plus qu'intéressante mais elles seraient sans doute hors sujet.

Toutefois, je n'ai pas trouvé le moyen de réaliser des graphiques en PHP, il est sûrement possible d'en réaliser comme j'ai pu constater sur PHP MyAdmin on nous parlait de création de graphique en bas des données des différentes tables. Malgré tout, étant en option math 8h en secondaire, j'ai déjà pu avoir connaissance de la programmation/création de graphiques statistiques grâce à la programmation en R. Peut-être qu'il y a une possibilité de liaison entre ses 2 types de programmations, ce qui les rendraient plutôt complémentaires.

Enfin, je pensais que le logiciel Microsoft Excel était puissant mais il n'est « rien* » comparé à l'entendue de développement de PHP. Certes bien plus compliqué, mais beaucoup plus complet d'après mon simple avis.

* Je ne dis pas que le logiciel Excel n'est pas intéressant mais après avoir pris connaissance de la programmation en PHP/MySQL, Excel devient un outil obsolète.

3) Phase de commencement :

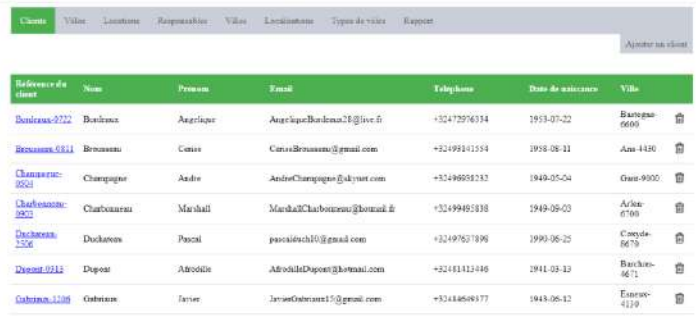
3.1) La base CSS

Le CSS est ce qui permet de structurer l'ensemble des différentes pages, sans oublier de l'ajouter sur chacune des pages au début du code, comme on peut le voir ci-dessous.

```
<!DOCTYPE html>
<html>
<meta charset="UTF-8">
<link rel="stylesheet" type="text/css" href="./style.css">
```

Il est plus que cruciale de l'ajouter sinon la page perd de sa structure prédéfinie. Un exemple ci-dessous avec la page d'index (Clients) :

Avec CSS



Sans CSS



On peut facilement remarquer que le CSS est là pour faciliter la lisibilité générale du système en structurant le menu ainsi qu'en ajoutant des couleurs et modifiant la police. Ce qui rend la lecture plus confortable pour l'utilisateur.

Le CSS que j'ai utilisé est semblable à celui utilisé dans les TP (Prof-Cours). Je l'ai trouvé plutôt simple et pratique.

Cependant, j'ai trouvé différents CSS en open source qui me semblaient plus complexes et plus esthétiques (https://getbootstrap.com/docs/4.4/examples/) mais j'ai préféré ne pas les utiliser pour rester dans le cadre du cours.

3.2) Les bases HTML

Avant de commencer la programmation en PHP il a fallu établir une base de départ pour le système, pour la navigation entre les différentes tables. Pour cela il a fallu créer un menu assez basique semblable à celui réaliser ultérieurement dans le devoir à rendre en mars.

```
<div class="contenant">
    <div class="menu">
        <ul class="menu">
            <li class="menu"><a class="active" href="index.php">Clients</a></li>
            <li class="menu"><a href="velo.php">Vélos</a></li>
            <li class="menu"><a href="location.php">Locations</a></li>
            <li class="menu"><a href="responsable.php">Responsables</a></li>
            <li class="menu"><a href="ville.php">Villes</a></li>
            <li class="menu"><a href="localisation.php">Localisations</a></li>
            <li class="menu"><a href="type_velo.php">Types de vélos</a></li>
            <li class="menu"><a href="rapport.php">Rapport</a></li>
        </ul>
    </div>
```

Après une sauvegarde sur les serveurs, grâce à FileZilla, on obtient concrètement ceci :



Cela permet de passer d'une table à l'autre de manière aisée et sans oublier de spécifier dans le code la page active par **« class="active" »**.

Ensuite, il a fallu créer plusieurs pages dédiées à l'ajout de données. Dans un premier temps nous verrons la base HTML avant de passer à la base PHP. Pour cela, je vais prendre pour exemple l'ajout d'une ville. Voici la partie du code :

```
<div>
<h2 align ="center"> Nouvelle ville</h2>

<form name="ville_insert" action="ajouter_ville.php" " method="post">
<table align="center" border=0>

<tr><td align=right>Référence de la ville</td><td><input type="text" name="id_ville" placeholder="Nom de la ville-code postal"></td></tr>
<tr><td align=right>Code Postal</td><td><input type="text" name="code_postal"></td></tr>
<tr><td align=right>Nom de la ville</td><td><input type="text" name="nom_ville"></td></tr>

<tr><td align=right><input type="submit" name="sub_btn" value="Ajouter"></td><td><input type="reset" name="res_btn" value="Réinitialiser"></td></tr>
</form>
```

Après le passage sur serveur ➔



J'ai trouvé le « placeholder » assez utile pour plusieurs pages d'ajout car il permet de donner une information supplémentaire à l'utilisateur pour faciliter la compréhension de la tâche à réaliser. Comme par exemple ici, pour la référence de la ville, qui est plutôt important de bien exécuter car comme on va le voir dans la partie PHP, la référence de la ville (id_ville) est une clé primaire de la table 'Ville' donc l'erreur est impérativement à éviter.

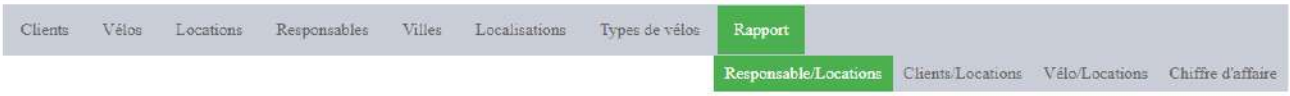
Nous parlerons aussi du champ « name=''….'' » dans la partie PHP, mais ici il est utilisé pour envoyer les données saisies dans la table 'Ville'.

Enfin, nous avons précédemment parlé du menu général au début de la page 6, maintenant nous allons voir le sous-menu qui est essentiellement utilisé pour l'ajout de données ou bien dans la section « rapport » qui sera ici, utilisé comme exemple. Voici le code :

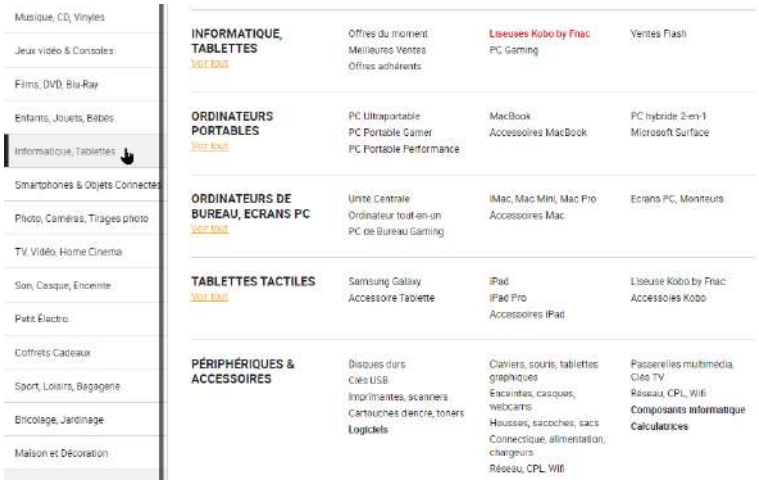
```
<div class="contenant">
    <div class="menu">
        <ul class="menu">
            <li class="menu"><a href="index.php">Clients</a></li>
            <li class="menu"><a href="velo.php">Vélos</a></li>
            <li class="menu"><a href="location.php">Locations</a></li>
            <li class="menu"><a href="responsable.php">Responsables</a></li>
            <li class="menu"><a href="ville.php">Villes</a></li>
            <li class="menu"><a href="localisation.php">Localisations</a></li>
            <li class="menu"><a href="type_velo.php">Types de vélos</a></li>
            <li class="menu"><a class="active" href="rapport.php">Rapport</a></li>
        </ul>

        <ul class="menu_s">
            <li class="menu_s"><a href="rapport3.php">Chiffre d'affaire</a></li>
            <li class="menu_s"><a href="rapport2.php">Vélo/Locations</a></li>
            <li class="menu_s"><a href="rapport1.php">Clients/Locations</a></li>
            <li class="menu_s"><a class="active" href="rapport.php">Responsable/Locations</a></li>
        </ul>
    </div>
```

La partie qui nous intéresse ici est la partie entourée en blanc avec le champ « menu_s ».
Après le passage sur serveur :



L'utilisation du sous-menu est très utilisé sur internet (en plus développé) sur les e-commerces comme par exemple sur le site de la Fnac.be :



N.B. : Pour afficher ce genre de menu, il est nécessaire d'utiliser un script Java.

Pour conclure avec la partie HTML, je dirais qu'elle est essentielle pour la liaison entre le système et la base de données sur PHP MyAdmin. C'est pourquoi il est important de la rédiger rigoureusement pour éviter les incoordinations.

4) La programmation :

4.1) La base de données

Tout d'abord, avant de commencer la programmation, il est important de créer une base de données pour enregistrer les informations qui seront recueillies grâce au système PHP. La base de données est sur « PHP MyAdmin », serveur : localhost et la base de données : mschyns_s160180. Les tables sont les suivantes : Client, Localisation, Location, Responsable, Type_velo, Velo et Ville. Elles possèdent toute comme interclassement : utf8_general_ci et ont entre 15 et 33 lignes (qui sont les données enregistrées pour chaque table).

Table	Action	Lignes	Type	Interclassement	Taille	Perte
Client	Afficher Structure Rechercher Insérer Vider Supprimer	30	InnoDB	utf8_general_ci	32 Kio	-
Localisation	Afficher Structure Rechercher Insérer Vider Supprimer	15	InnoDB	utf8_general_ci	32 Kio	-
Location	Afficher Structure Rechercher Insérer Vider Supprimer	33	InnoDB	utf8_general_ci	32 Kio	-
Responsable	Afficher Structure Rechercher Insérer Vider Supprimer	15	InnoDB	utf8_general_ci	16 Kio	-
Type_velo	Afficher Structure Rechercher Insérer Vider Supprimer	17	InnoDB	utf8_general_ci	16 Kio	-
Velo	Afficher Structure Rechercher Insérer Vider Supprimer	29	InnoDB	utf8_general_ci	48 Kio	-
Ville	Afficher Structure Rechercher Insérer Vider Supprimer	30	InnoDB	utf8_general_ci	16 Kio	-
7 tables	Somme	169	InnoDB	latin1_swedish_ci	192 Kio	0 o

Je vais ensuite détailler chacune de ses tables l'une après l'autre en parlant de leur structure (attributs, type, clé primaire et clé secondaire) ainsi que leurs relations entre elles.

* Client :

#	Nom	Type	Interclassement	Attributs	Null	Défaut	Extra
1	id_client	varchar(20)	utf8_general_ci		Non	*Aucune*	
2	nom	varchar(50)	utf8_general_ci		Non	*Aucune*	
3	prenom	varchar(50)	utf8_general_ci		Non	*Aucune*	
4	email	varchar(50)	utf8_general_ci		Non	*Aucune*	
5	telephone	varchar(15)	utf8_general_ci		Non	*Aucune*	
6	date_naissance	date			Non	*Aucune*	
7	id_ville	varchar(20)	utf8_general_ci		Non	*Aucune*	

Les attributs de la table 'Client' sont donc id_client (qui est la clé primaire de la table), nom, prenom, email, telephone, date_naissance et id_ville (qui est la clé étrangère).

J'ai utilisé VARCHAR et DATE comme type pour cette table. Après de multiples erreurs avec INT, j'ai décidé d'utiliser VARCHAR dans la majorité de mes tables pour faciliter l'encodage et éviter des erreurs.

La table ‘Client’ est en relation avec la table ‘Ville’ comme on peut le voir ci-dessous dans les contraintes de clé étrangère :



Cela signifie concrètement qu’une ville peut être associée à plusieurs clients, ce qui est normal. Voici un bref échantillon de la table ‘Client’ (l’intégralité des tables seront communiquées en annexe) :

id_client	nom	prenom	email	telephone	date_naissance	id_ville
Bordeaux-0722	Bordeaux	Angelique	AngeliqueBordeaux28@live.fr	+32472976334	1953-07-22	Bastogne-6600
Brousseau-0811	Brousseau	Cerise	CeriseBrousseau@gmail.com	+32493141554	1958-08-11	Ans-4430
Champagne-0504	Champagne	Andre	AndreChampagne@skynet.com	+32496938232	1949-05-04	Gant-9000
Charbonneau-0903	Charbonneau	Marshall	MarshallCharbonneau@hotmail.fr	+32499495838	1949-09-03	Arlon-6700
Duchateau-2506	Duchateau	Pascal	pascalduch10@gmail.com	+32497637898	1990-06-25	Coxyde-8670

* Localisation :

#	Nom	Type	Interclassement	Attributs	Null	Défaut	Extra
1	id_localisation	varchar(20)	utf8_general_ci		Non	*Aucune*	
2	nom_loc	varchar(50)	utf8_general_ci		Non	*Aucune*	
3	description	varchar(200)	utf8_general_ci		Non	*Aucune*	
4	id_ville	varchar(20)	utf8_general_ci		Non	*Aucune*	

Les attributs de la table ‘Localisation’ sont donc id_localisation (qui est la clé primaire de la table), nom_loc, description et id_ville (qui est la clé étrangère).

Le type utilisé pour cette table est VARCHAR.

La table ‘Localisation’ est en lien avec la table ‘Responsable’ et ‘Ville’ comme on peut le voir ci-dessous :



C'est-à-dire qu'un responsable peut être associé à une seule localisation et vice-versa (liaison de 2 clés primaire-relation one to one). Donc la référence pour un responsable et une localisation seront identique car la localisation n°1 correspondra uniquement au responsable X (id_localisation = id_responsable). Et une ville peut être associée à plusieurs localisations. Voici un bref exemple de la table 'Localisation' :

id_localisation	nom_loc	description	id_ville
Artois-1906	Charleroi Airport	Aeroport	Charleroi-6000
Bernard-0507	Gare de Flemalle Haute	Gare	Flemalle-4400
Blanchard-0809	Gare de Gembloux	Gare	Gembloux-5030
Blondlot-1303	Hotel sur la cote belge	Hotel	Knokke-8300
Bouvier-1806	Gare de Louvain la Neuve	Gare	Ottignies-1340

* Location :

#	Nom	Type	Interclassement	Attributs	Null	Défaut	Extra
1	id_location	varchar(20)	utf8_general_ci		Non	*Aucune*	
2	date_location	date			Non	*Aucune*	
3	duree_location	varchar(3)	utf8_general_ci		Non	*Aucune*	
4	prix_location	varchar(3)	utf8_general_ci		Non	*Aucune*	
5	id_client	varchar(20)	utf8_general_ci		Non	*Aucune*	
6	id_velo	varchar(20)	utf8_general_ci		Non	*Aucune*	
7	id_responsable	varchar(20)	utf8_general_ci		Non	*Aucune*	

Les attributs de la table 'Location' sont donc id_location, date_location (clé primaire), duree_location, prix_location, id_client (clé étrangère), id_velo (clé primaire) et id_responsable (clé étrangère).

Les type utilisés pour cette table sont VARCHAR et DATE.

Pour cette table, qui est sans doute celle qui lie toutes les tables entre elles, j'ai choisi 2 clés primaire « date_location » et « id_velo » car le même vélo ne peut pas être loué le même jour. Voici les relations :



Il n'y a pas de contraintes pour les clés id_client et id_responsable car un client peut avoir plusieurs location et un responsable peut avoir aussi plusieurs locations à son actif.

Voici un bref extrait de la table 'Location' :

id_location	date_location	duree_location	prix_location	id_client	id_velo	id_responsable
BE-0008	2020-04-24	2	100	Gervais-0217	VELO-003-50	Dupont-0511
BE-0009	2020-04-25	3	90	Guernon-2704	VELO-007-30	Bouvier-1806
BE-0004	2020-04-28	1	30	Dupont-0313	VELO-027-30	Artois-1906
BE-0001	2020-05-10	2	50	Ricard-2205	VELO-001-25	Dumoulin-1005

* Responsable :

#	Nom	Type	Interclassement	Attributs	Null	Défaut	Extra
1	id_responsable	varchar(20)	utf8_general_ci		Non	*Aucune*	
2	nom	varchar(50)	utf8_general_ci		Non	*Aucune*	
3	prenom	varchar(50)	utf8_general_ci		Non	*Aucune*	
4	email	varchar(50)	utf8_general_ci		Non	*Aucune*	
5	telephone	varchar(15)	utf8_general_ci		Non	*Aucune*	
6	date_naissance	date			Non	*Aucune*	

Les attributs de la table 'Responsable' sont donc id_responsable (clé primaire), nom, prenom, email, telephone et date_naissance.

Les types utilisés pour cette table sont VARCHAR et DATE.

La table 'Responsable' est en relation avec la table 'Location' et 'Localisation' comme on a pu le voir ci-dessus. Voir un bref échantillon de la table 'Responsable' :

id_responsable	nom	prenom	email	telephone	date_naissance
Artois-1906	Artois	Noemie	noemieartois@live.fr	+3248378335	1967-06-19
Bernard-0507	Bernard	Mathilde	bern.math@live.fr	+32472209451	1980-07-05
Blanchard-0809	Blanchard	Xavier	xavierblanchard100@hotmail.com	+32493371999	1988-09-08
Blondlot-1303	Blondlot	Jacques	jacques.blondlot58@gmail.com	+32496878754	1959-03-13

* Type_velo :

#	Nom	Type	Interclassement	Attributs	Null	Défaut	Extra
1	id_type_velo	varchar(50)	utf8_general_ci		Non	*Aucune*	
2	type	varchar(50)	utf8_general_ci		Non	*Aucune*	

Les attributs de la table 'Type_velo' sont donc id_type_velo (clé primaire) et type.

Le type utilisé pour cette table est VARCHAR.

La seule relation que cette table possède c'est avec la table 'Velo' que nous verrons ci-dessous. Voici un extrait de la table 'Type_velo' :

id_type_velo	type
BMX	TYP-013
Cargo	TYP-008
Couche(2roues)	TYP-011
Couche(3roues)	TYP-012
Course	TYP-001
Electrique	TYP-005

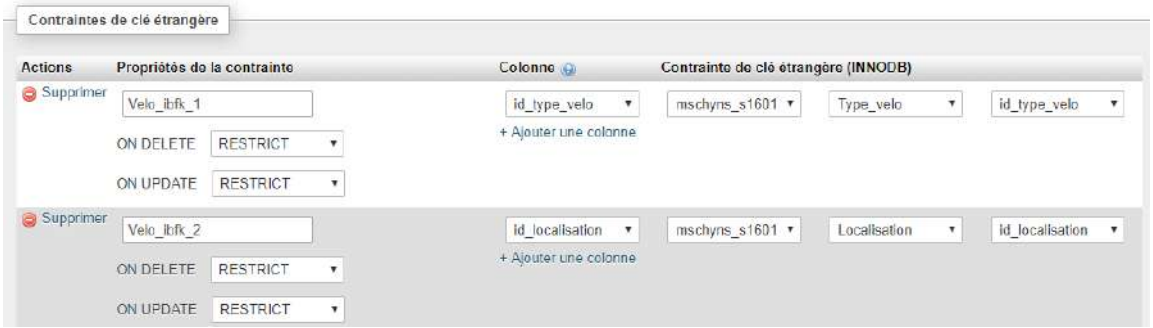
* Velo :

#	Nom	Type	Interclassement	Attributs	Null	Défaut	Extra
1	id_velo	varchar(50)	utf8_general_ci		Non	*Aucune*	
2	date_achat	date			Non	*Aucune*	
3	prix_velo	varchar(20)	utf8_general_ci		Non	*Aucune*	
4	description	varchar(250)	utf8_general_ci		Non	*Aucune*	
5	prix_location	varchar(20)	utf8_general_ci		Non	*Aucune*	
6	id_type_velo	varchar(50)	utf8_general_ci		Non	*Aucune*	
7	id_localisation	varchar(20)	utf8_general_ci		Non	*Aucune*	

Les attributs de la table 'Velo' sont donc id_velo (clé primaire), date_achat, prix_velo, description, prix_location, id_type_velo (clé étrangère) et id_localisation (clé étrangère).

Les types utilisés pour cette table sont VARCHAR et DATE.

La table 'Velo' est en relation avec la table 'Type_velo' et 'Localisation'. Voici les contraintes de clé étrangère :



C'est-à-dire qu'il n'y a qu'un seul type de vélo pour un vélo et qu'un vélo ne concerne qu'une seule localisation.

Voici un extrait de la table ‘Velo’ :

id_velo	date_achat	prix_velo	description	prix_location	id_type_velo	id_localisation
VELO-001-25	2019-05-20	1300	Velo taille moyenne	25	Couche(2roues)	Dumoulin-1005
VELO-002-10	2017-10-05	900	Velo petite taille	10	Pliant	Gilbert-1005
VELO-003-50	2012-05-26	2500	Velo grande taille	50	Electrique	Dupont-0511
VELO-004-50	2019-03-01	5000	Velo grande taille	50	Course	Lamoureux-2504

* Ville :

#	Nom	Type	Interclassement	Attributs	Null	Défaut	Extra
1	id_ville	varchar(20)	utf8_general_ci		Non	*Aucune*	
2	code_postal	varchar(4)	utf8_general_ci		Non	*Aucune*	
3	nom_ville	varchar(25)	utf8_general_ci		Non	*Aucune*	

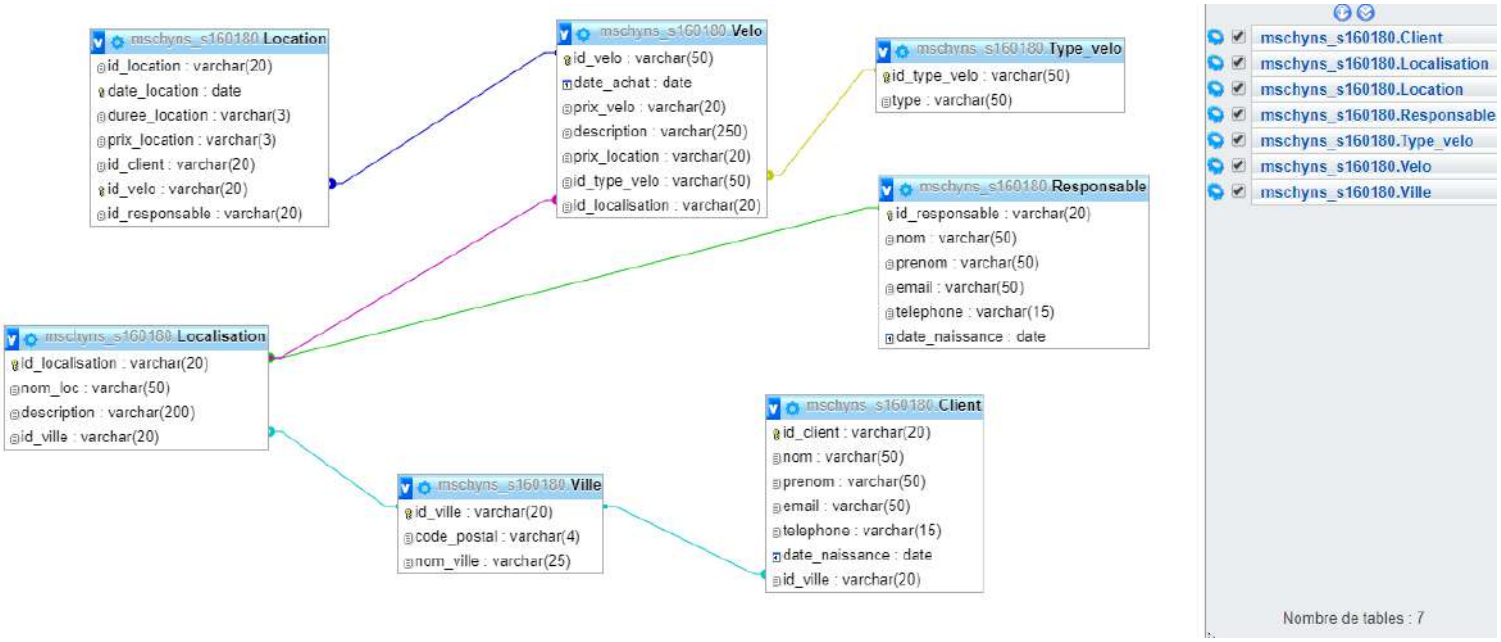
Les attributs de la table ‘Ville’ sont donc id_ville (clé primaire), code_postal et nom_ville.

Le type utilisé pour cette table est VARCHAR.

La table ‘Ville’ est en relation avec la table ‘Client’ et ‘Localisation’. Voici un extrait de cette dernière :

id_ville	code_postal	nom_ville
Amay-4540	4540	Amay
Andenne-5300	5300	Andenne
Angleur-4031	4031	Angleur
Ans-4430	4430	Ans

Pour finir, voici l'ensemble des relations des différentes tables émanant du concepteur sur PHP MyAdmin :



4.2) Liaison PHP/MySQL

Avant de parler de la programmation en PHP il est plus qu'important de se pencher sur la liaison entre la base de données et le programme.

Lorsque les différentes tables furent complétées, il a bien fallu les associer aux différentes pages pour qu'elles puissent communiquer entre elles.

Pour cela, le schéma était plutôt identique pour l'ensemble des pages. Voici ci-dessous le code qui est présent sur chaque page :

```php
<?php
    $db_user = "s160180";
    $db= "mschyns_s160180";
    $db_pass = "mrhxz46474";
    $con= mysqli_connect("php2.hec.ulg.ac.be",$db_user,$db_pass,$db);

    if(mysqli_connect_errno()){
    echo "Failed to connect to MySQL: ".mysqli_connect_error();
    }
```

Les 4 premières lignes sont l'outil de liaison ($db_user, $db, $db_pass et $con) vers la base de données.

Les 3 derniers lignes sont là pour notifier l'utilisateur d'une éventuelle erreur dans la connexion avec MySQL grâce à la fonction « if* ».

* La fonction if(..) signifie que SI ce qui est entre parenthèse arrive elle affichera un message grâce à « echo ».

Ensuite, après ceci sont suivis une requête et un résultat à afficher. Je vais prendre pour exemple la page d'index qui est l'ensemble des clients.

Je souhaite donc réaliser un tableau composé de l'ensemble des clients de la base de données avec diverses informations telle que id_client, nom, prenom, email, telephone, date_naissance et id_ville de la table 'Client'. Voici la requête et le résultat :

```
$requete = "SELECT id_client, nom, prenom, email, telephone, date_naissance, id_ville FROM Client";
$result = mysqli_query($con,$requete);
```

Après, 2 « echo » sont nécessaires à la structure du tableau :

```
echo "<table class=\"list\">";
echo "<th>Référence du client</th><th>Nom</th><th>Prénom</th><th>Email</th><th>Téléphone</th><th>Date de naissance</th><th>Ville</th><th></th>";
```

Enfin, il faut réaliser une boucle qui permettra d'afficher l'ensemble des clients, sans cette dernière il n'y aura qu'un seul client affiché. La boucle utilisée est « while » suivi des « echo » des différents attributs de la table sélectionné depuis la requête.

```
while($row = mysqli_fetch_array($result)){
    echo "<tr class=\"list\">";
    echo "<td class=\"list\" width=\"10%\" ><a href=update_client.php?id_client=".$row['id_client'].">".$row['id_client']."</a></td>";
    echo "<td class=\"list\" width=\"15%\">".$row['nom']."</td>";
    echo "<td class=\"list\" width=\"15%\">".$row['prenom']."</td>";
    echo "<td class=\"list\" width=\"25%\">".$row['email']."</td>";
    echo "<td class=\"list\" width=\"15%\">".$row['telephone']."</td>";
    echo "<td class=\"list\" width=\"15%\">".$row['date_naissance']."</td>";
    echo "<td class=\"list\" width=\"10%\">".$row['id_ville']."</td>";
    echo "<td class=\"list\" width=\"5%\"><a href=supprimer_client.php?id_client=".$row['id_client']."><img src=\"trash.png\" alt=\"del\" height=\"30\"></a></td></tr>";
    echo "</tr>";
}
echo "</table>";
```

N.B. : Pour la ligne id_client, il s'agit d'un hyper lien permettant la modification qui sera vue plus tard.

Idem pour la dernière ligne de suppression de id_client.

Il est important de mettre l'ensemble des attributs sélectionné dans la requête en echo sinon on aboutit à une erreur « mysli_fetch_array_error ».

Et pour finir, il faut fermer la connexion MySQL (ainsi PHP avec ?>) :

```
        mysqli_close($con);
    ?>
</body>
</html>
```

4.3) Les 3 applications du programme : ajouter, modifier et supprimer

*Ajouter un élément :

Je vais continuer sur mon exemple d'ajout d'une ville de la section HTML. Voici le code complet avec les explications par partie :

```php
<?php
    if (isset($_POST['id_ville'])){
    $id_ville= $_POST["id_ville"];
    $code_postal= $_POST["code_postal"];
    $nom_ville= $_POST["nom_ville"];;

    if($_POST["id_ville"] == ''|| $_POST["code_postal"] == ''|| $_POST["nom_ville"] == ''){
        echo "<center class=\"alert\">Référence de la ville, code postal ou le nom de la ville ne peuvent pas être vides.</center>";
    }else{
        $db_user = "s160180";
        $db= "mschyns_s160180";
        $db_pass = "mrhxz46474";
        $con= mysqli_connect("php2.hec.ulg.ac.be",$db_user,$db_pass,$db);

        if (mysqli_connect_errno()){
        echo "Failed to connect to MySQL: " . mysqli_connect_error();
        }
        if (mysqli_query($con,"INSERT INTO Ville (id_ville, code_postal, nom_ville) VALUES ('$id_ville', '$code_postal','$nom_ville')")){
            echo "<center>Ville rajouté dans la base de données</center> <br>";
        }else{
            echo '<center>'.mysqli_error($con).'</center> <br>';
        }


        mysqli_close($con);
    }
}

?>
```

➔ Pour la partie rouge :

Il s'agit de recueillir les données encodées par l'utilisateur pour id_ville, code_postal et nom_ville. La fonction « isset » est utilisée pour vérifier que la variable id_ville, qui est la clé primaire, ne soit pas NULL.

➔ Pour la partie bleue :

Elle vérifie que les 3 champs obligatoires ont bien été remplis, dans le cas contraire elle renverra « echo » (message d'erreur/alerte en rouge) :

Nouvelle ville

Référence de la ville, code postal ou le nom de la ville ne peuvent pas être vides.

Référence de la ville: Nom de la ville-code postal
Code Postal:
Nom de la ville:
Ajouter | Réinitialiser

➔ Pour la partie verte :

Cette partie permet la liaison avec la base de données, comme nous venons de le voir ci-dessus.

➔ Pour la partie blanche :

Elle est essentielle pour l'ajout des valeurs $id_ville, $code_postal et $nom_ville dans la table 'Ville' grâce à « INSERT INTO ». Si toutes les données sont valides, elle renverra un message de confirmation (avec « echo »).

➔ Pour la partie brune :

Il s'agit de la partie de fermeture comme nous venons de le voir ci-dessus.

* Modifier un élément :

Continuons sur la table 'Ville' :

```php
<?php
if (isset($_GET['id_ville'])){

    $db_user = "s160180";
    $db= "mschyns_s160180";
    $db_pass = "mrhxz46474";
    $con= mysqli_connect("php2.hec.ulg.ac.be",$db_user,$db_pass,$db);

    if (mysqli_connect_errno()){
        echo "Failed to connect to MySQL: " . mysqli_connect_error();
    }

    $id_ville = $_GET["id_ville"];
    $requete = "SELECT * FROM Ville WHERE id_ville = '" . $id_ville . "'";
    $result = mysqli_query($con, $requete);
    $nbLigne = mysqli_num_rows($result);

    if($nbLigne == 0){
        die("<center class=\"alert\">Cette ville '.$id_ville.' est inexistant.</center>");
    }else{
        $row = mysqli_fetch_array($result);
        $code_postal=$row['code_postal'];
        $nom_ville=$row['nom_ville'];
    }

    if (isset($_POST["save_btn"])){
        $code_postal=$_POST["code_postal"];
        $nom_ville=$_POST["nom_ville"];

        if($_POST["code_postal"] == ''|| $_POST["nom_ville"] == ''){
            echo "<center>Code postal ou nom de la ville ne peuvent pas être vides.</center>";
        }else{
            if (!mysqli_query($con,"UPDATE Ville SET code_postal='".$code_postal."',nom_ville='".$nom_ville."' WHERE id_ville = '".$id_ville."'")){
                die('Erreur: ' . mysqli_error($con));
            }
            echo "<center>Cette ville ".$id_ville." est modifié dans la base de données</center>";
        }
    }
```

➔ Partie en rouge :

Liaison avec base de données MySQL.

➔ Partie en bleu :

Recherche d'une correspondance pour la ville sélectionnée dans la table 'Ville'.

➔ Partie en vert :

Nous informe soit que la ville est inexistante soit qu'elle existe et les données sont prise en considération

➔ Partie en blanc :

Le code nous indique que les 2 champs code_postal et nom_ville ne peuvent pas être vides. Dans le cas où toutes les données sont valables la modification aura lieu dans la base de données grâce à « UPDATE ».

Ensuite, il modifiera les données grâce aux « echo » suivent :

```
echo "<form name=\"ville_update\" action=\"update_ville.php?id_ville=$id_ville\" method=\"post\">";
echo "<table align=\"center\">";
echo "<tr><td align=right>Référence de la ville: </td><td>".$id_ville."<input type=\"hidden\" name=\"id_ville\" value=\"".$id_ville."\"></td></tr>";
echo "<tr><td align=right>Code Postal: </td><td><input type=\"text\" name=\"code_postal\" value=\"".$code_postal."\"></td></tr>";
echo "<tr><td align=right>Nom de la ville: </td><td><input type=\"text\" name=\"nom_ville\" value=\"".$nom_ville."\"></td></tr>";
echo "</td></tr>";
echo "<tr><td align=right><input type=\"submit\" name=\"save_btn\" value=\"Modifier\"></td><td><input type=\"reset\" name=\"res_btn\" value=\"Réinitialiser\"></td></tr>";
echo "</table>";
echo "</form>";
```

Pour ce cas, id_ville ne peut pas être modifié car il est déjà lié avec un ou plusieurs client(s).

Enfin, un tableau représentant l'ensemble des clients lié à la ville sélectionné est disponible grâce au code suivant :

```
$result_ville = mysqli_query($con, "SELECT id_client FROM Client WHERE id_ville = '" . $id_ville . "'");
echo "<table class=\"list\" align=\"center\">";
echo "<th>Références des Clients de cette ville</th>";
while($row_ville = mysqli_fetch_array($result_ville)){
    echo "<tr class=\"list\">";
    echo "<td class=\"list\" width=\"100%\">".$row_ville['id_client']."</a> </td>";
    echo "</tr>";
}
echo "</table>";
```

Sans oublier de fermer la connexion au serveur ainsi que le lieu où on modifie la ville grâce à « header ».

```
    mysqli_close($con);

}else{
    header('Location: ville.php');
    exit();
}
?>
```

➔

```
><a href=update_ville.php?id_ville=".$row['id_ville'].">".$row['id_ville']."</a></td>";
```

* Supprimer un élément :

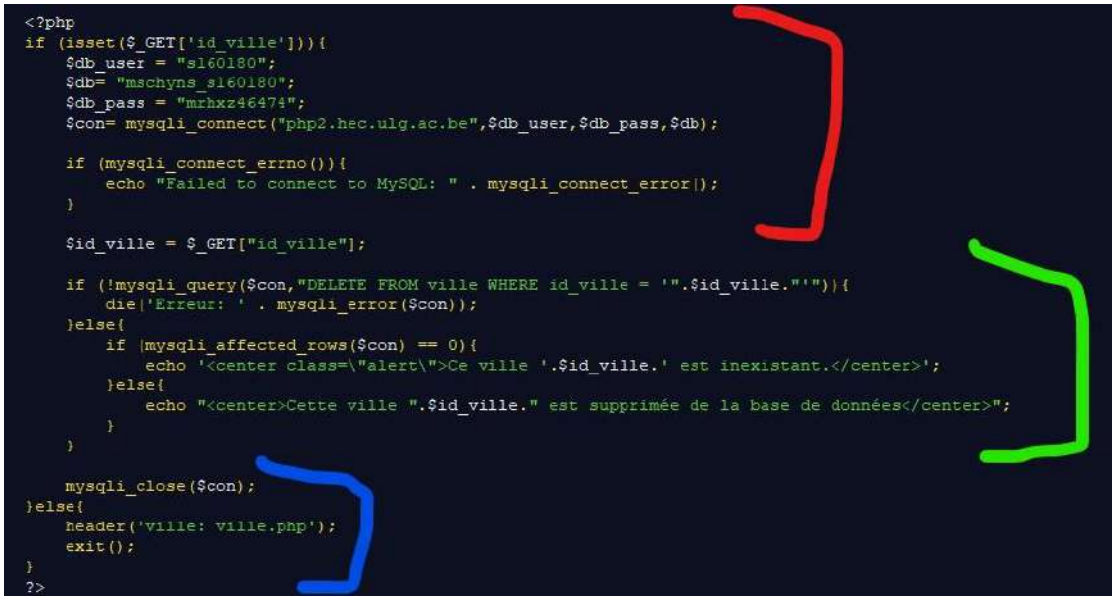
Le lien pour supprimer un élément est en cliquant sur la poubelle dans la base de données :

Référence de la ville	Code postal	Nom de la ville	
Amay-4540	4540	Amay	

Et ceci grâce à l' « echo » suivant :

```
echo "<td class=\"list\" width=\"5%\"><a href=supprimer_ville.php?id_ville=".$row['id_ville']."><
    <img src=\"trash.png\" alt=\"del\" height=\"30\"></a></td></tr>";
```

Voyons maintenant le code de suppression d'une ville :



```php
<?php
if (isset($_GET['id_ville'])){
    $db_user = "s160180";
    $db= "mschyns_s160180";
    $db_pass = "mrhxz46474";
    $con= mysqli_connect("php2.hec.ulg.ac.be",$db_user,$db_pass,$db);

    if (mysqli_connect_errno()){
        echo "Failed to connect to MySQL: " . mysqli_connect_error();
    }

    $id_ville = $_GET["id_ville"];

    if (!mysqli_query($con,"DELETE FROM ville WHERE id_ville = '".$id_ville."'")){
        die('Erreur: ' . mysqli_error($con));
    }else{
        if (mysqli_affected_rows($con) == 0){
            echo '<center class=\"alert\">Ce ville '.$id_ville.' est inexistant.</center>';
        }else{
            echo "<center>Cette ville ".$id_ville." est supprimée de la base de données</center>";
        }
    }

    mysqli_close($con);
}else{
    header('ville: ville.php');
    exit();
}
?>
```

➔ Partie en rouge :

Liaison avec la base de données MySQL qui correspond à l'id_ville sélectionné.

➔ Partie en vert :

Suppression de la ville sélectionnée grâce à « DELETE » si la ville n'existe pas « echo » renverra que la ville n'existe pas et si la ville sélectionnée existe, elle sera supprimée.

➔ Partie en bleu :

Fermeture de la connexion avec le serveur ainsi que la localisation du lien de suppression vu ci-dessus.

5) Les analyses statistiques : Explications

La première analyse statistique concerne le nombre de location ainsi que la recette pour chaque responsable dans l'ordre alphabétique (il est toutefois possible de l'ordonner différemment). Cela permet de voir les performances individuelles pour chaque responsable. Par exemple dans notre cas, on peut voir que le responsable « ARTOIS Noemie » à 6 locations à venir et le responsable « DUBOIS Nicolas » n'en a que 2. On peut interpréter ça de plusieurs manières différentes mais partons du principe que chaque responsable on la même « productivité ». Il est possible que dans un futur proche la responsable Madame Artois soit débordée par le nombre de locations et nous pourrions assigner le responsable Monsieur Dubois avec Madame Artois pour une meilleure répartition des effectifs.

On peut aussi voir la recette pour chaque responsable et à partir de ça on pourrait éventuellement assigner un bonus pour le responsable qui effectue la meilleure recette du mois.

Pour effectuer les statistiques ci-dessus, j'ai utilisé les requêtes « count » et « sum ».

Ensuite, la seconde analyse statistique concerne le rapport entre les clients et les locations. On peut y trouver le nombre de locations par client, le nombre de jour(s) de location total ainsi que le prix total des locations pour chacun d'eux.

Grâce à ceci, on peut voir d'un coup d'œil (s'il n'y a pas des centaines de locations) les clients qui ont effectué le plus de locations. Dans notre cas, on peut voir que le client « RICARD Travis » a réalisé 10 locations pour une durée totale de 35 jours. D'un point de vue marketing on pourrait le considérer comme un « client en or ». Il serait donc avantageux de le garder parmi nos clients et donc éventuellement lui proposer des réductions ou d'autres offres promotionnelles.

On peut aussi remarquer des clients de nature « unique/très occasionnel » il s'agit de ceux qui n'ont effectué qu'une seule location.

Pour la troisième analyse statistique, on met en relation les vélos et les locations pour voir les vélos les plus loués. On peut voir par exemple que le vélo « VELO-017-50 » a été loué 3 fois pour une recette totale de 400 euros. Lorsque nous aurons plus de locations on pourra plus facilement voir les vélos préférés des clients et améliorer la disponibilité de ces derniers en agrandissant leur stock.

J'ai aussi rajouté une dernière colonne contenant le prix du vélo comme ça on peut facilement voir quand le vélo sera totalement amorti et commencera à être rentable pour BikeRental.

Pour le dernier rapport, il concerne le chiffre d'affaires mensuel en fonction des dates de réservations entré dans la base de données. Comme il s'agit d'un nouveau système il est assez difficile d'interpréter les résultats mais au fur et à mesure des années on pourra voir les jours/mois avec le plus de demande et ainsi anticiper l'allocation des ressources (personnels et vélos). Cependant il permet de faciliter les comptes de la société pour rémunérer les 25% du aux différents responsables.

En théorie les statistiques doivent être réalisées sur plusieurs centaines de données pour être efficiente, c'est-à-dire que les rapports statistiques auront de plus en plus de valeur au cours des années à venir.

6) Conclusion du travail

Pour conclure ce travail, je pense bien avoir démontré l'éligibilité de mon système et de son fonctionnement sur le long terme.

Il est assez difficile de dire que la programmation en PHP est limitée car il y a des formules à n'en plus finir …

Cependant, cet outil est loin d'être simple à l'utilisation, voire très complexe dans un premier temps. Ayant rencontré de nombreuses erreurs lors de l'affichage en PHP, je peux confirmer que si on oublie le moindre caractère tel qu'une parenthèse, un crochet, un point-virgule ou bien une majuscule, plus rien ne s'affiche ou des messages d'erreurs où la source du problème est difficilement identifiable. Par exemple, j'ai eu quelques erreurs par rapport aux tables (oubli d'une majuscule ➔ … FROM ville), du coup rien ne s'envoyait et ce n'était pas une mince affaire pour retrouver l'erreur dans presque cent lignes de code.

Pour finir, je dirais que la programmation demande énormément de concentration et de vigilance dans chaque exécution de caractère.

7) Bibliographie

* Formation HTML, CSS et PHP en ligne (voir phase d'introduction).

* http://www.generatedata.com/?lang=fr#t1 (générateur de nom et adresse mail essentiellement utilisé pour la création de clients et responsables).

* https://www.php.net/ (ensemble des formules et explications PHP).

* https://www.youtube.com/results?search_query=php (visionnage de quelques vidéos explicatives concernant le codage en PHP).